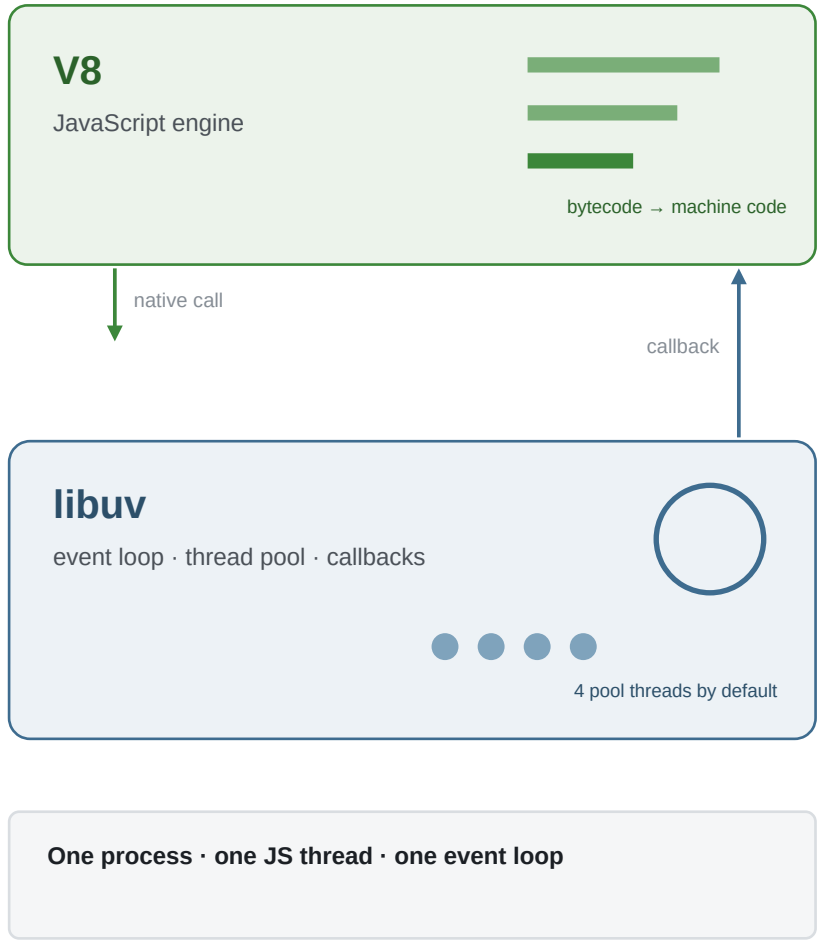
ELEVATE · WEEK 1 · SESSION 1 · PART 2

Node.js Under the Hood

V8, libuv, and the event loop

Part 2 of 2

What executes your JavaScript, what waits on the operating system — and how the two meet.



Runtime architecture → event loop → blocking → thread pool → workers

PART 2 · AGENDA

From primitives to runtime

PART 1 · ESTABLISHED

- ✓ **Process** — isolation, own memory
- ✓ **Thread** — execution, shared heap
- ✓ **Concurrency models** — process → thread → loop → hybrid

NOW WE OPEN THE RUNTIME

01
Node.js architecture
V8, Node core, libuv, OS

02
Who executes what
the handoff on every async call

03
Event loop in one picture
stack · pool · callback queue

04
CPU-bound vs I/O-bound
where the time actually goes

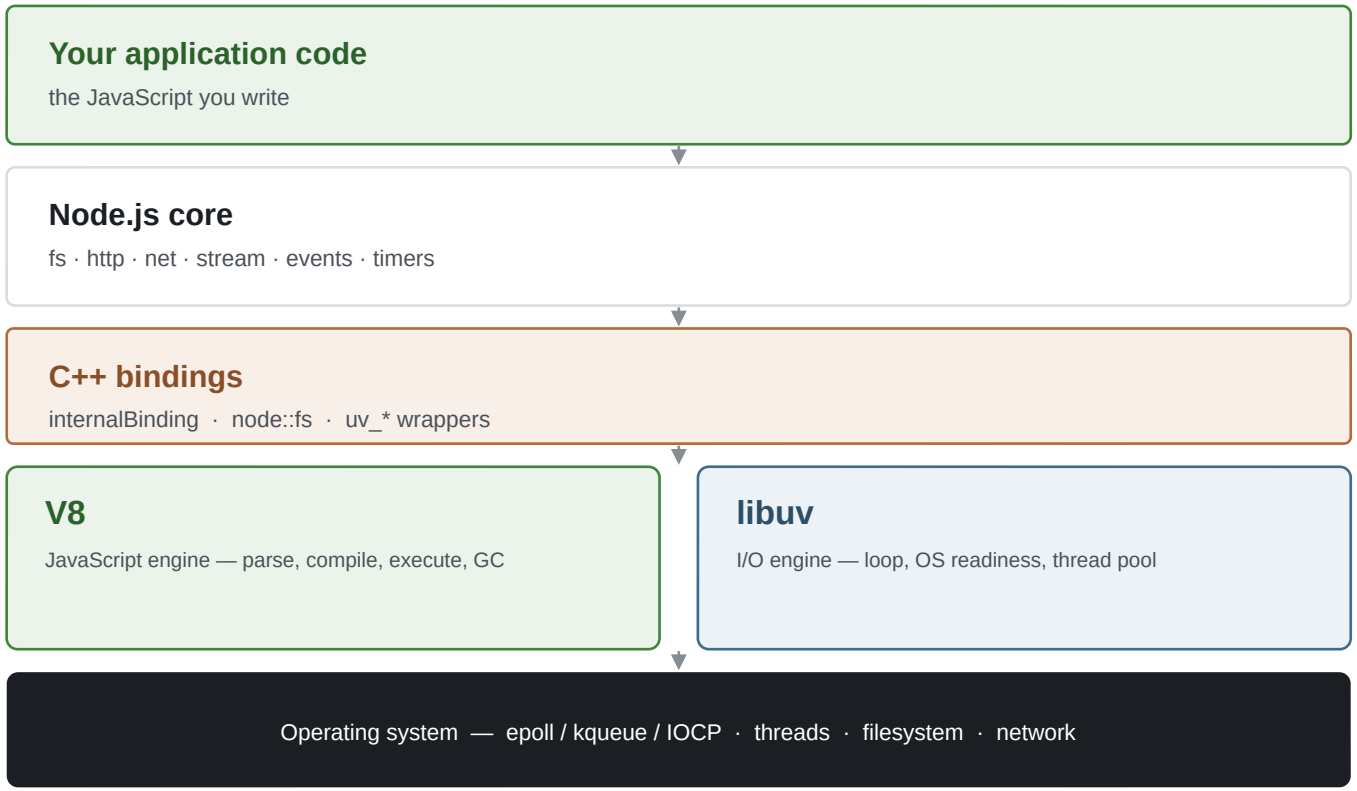
05
Blocking the loop
symptoms and measurement

06
Pool & worker threads
parallelism when you need it

By the end: you should know exactly which engine is doing the work at every await.

The Node.js runtime

ONE PROCESS



WHAT THE LAYERS MEAN

- Node core is mostly JavaScript, plus C++ bindings for the parts JS cannot do.
- Bindings are the only door from JS to the OS. They translate arguments and call libuv or V8.
- V8 and libuv are native libraries linked into the node binary — not separate processes.
- Your JavaScript never runs in libuv, and libuv never runs your callbacks.
- Every async call crosses this stack twice: once on the way out, once on the way back.

The OS is the only layer that can actually wait.

Adapted from Casciaro & Mammino, Node.js Design Patterns, 3rd ed. (2020), Fig. 1.4, p. 12.

RUNTIME · 02

Who executes what?

V8

JavaScript engine

- Parses and compiles JavaScript
- Runs your callbacks and expressions
- Call stack, heap, garbage collection
- Promise reactions (microtasks)
- Optimizes hot functions

IN PRACTICE

JSON.parse · regex · array.map

NODE CORE

API layer + bindings

- Public APIs: fs, http, net, stream
- Module loader and EventEmitter
- process.nextTick queue
- Wraps C++ bindings for JS
- Chooses libuv or a direct syscall

IN PRACTICE

fs.readFile · http.createServer

LIBUV

I/O engine

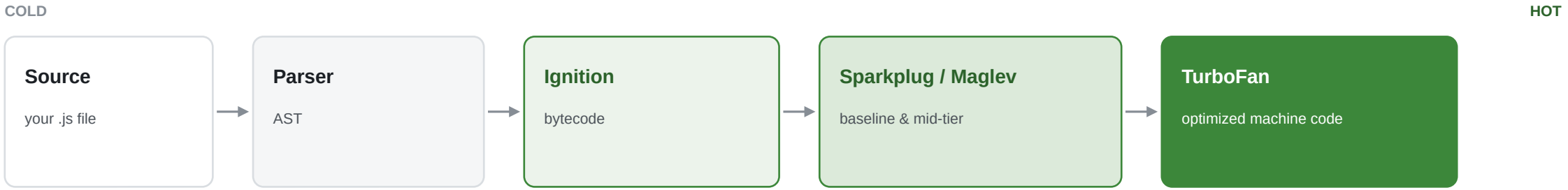
- Event loop and callback dispatch
- OS readiness: epoll / kqueue / IOCP
- Thread pool (4 by default)
- fs, dns.lookup, crypto, zlib
- Timers and cross-platform I/O

IN PRACTICE

epoll_wait · pread · getaddrinfo

V8 runs JavaScript. libuv runs the wait. Node core is the glue between them.

Inside V8: source to machine code



Every function starts in Ignition. Type feedback decides what gets optimized.

Ignition

Interprets bytecode immediately. No warm-up, no machine code yet.

Tier up

Hot functions compile to native code. A failed assumption deopts back to bytecode.

Heap & GC

Objects live in V8's heap. Collection pauses are part of your latency budget.

V8 never opens a file, reads a socket, or waits on a timer. Its job ends at executing JavaScript.

Tiering details: V8 docs — Ignition, Sparkplug, Maglev, TurboFan.

What V8 actually executes

terminal

```
$ node --print-bytecode -e "const add=(a,b)=>a+b; add(2,3)"

[generated bytecode for function: add]
Parameter count 3
Register count 0
  0 : Ldar a1
  2 : Add a0, [0]
  5 : Return
Constant pool (size = 0)
```

Actual output: Node 24.21 · V8 13.6 · addresses trimmed for readability.

Ldar a1

load argument b

Add a0, [0]

add argument a

Return

hand the result back

1 V8

Parse once

The arrow function is parsed and compiled before add(2, 3) runs.

2 IGNITION

Emit bytecode

Three instructions: not machine code, and not C++.

3 V8

Execute

Ldar loads b, Add adds a, Return hands the result back to the caller.

4 V8

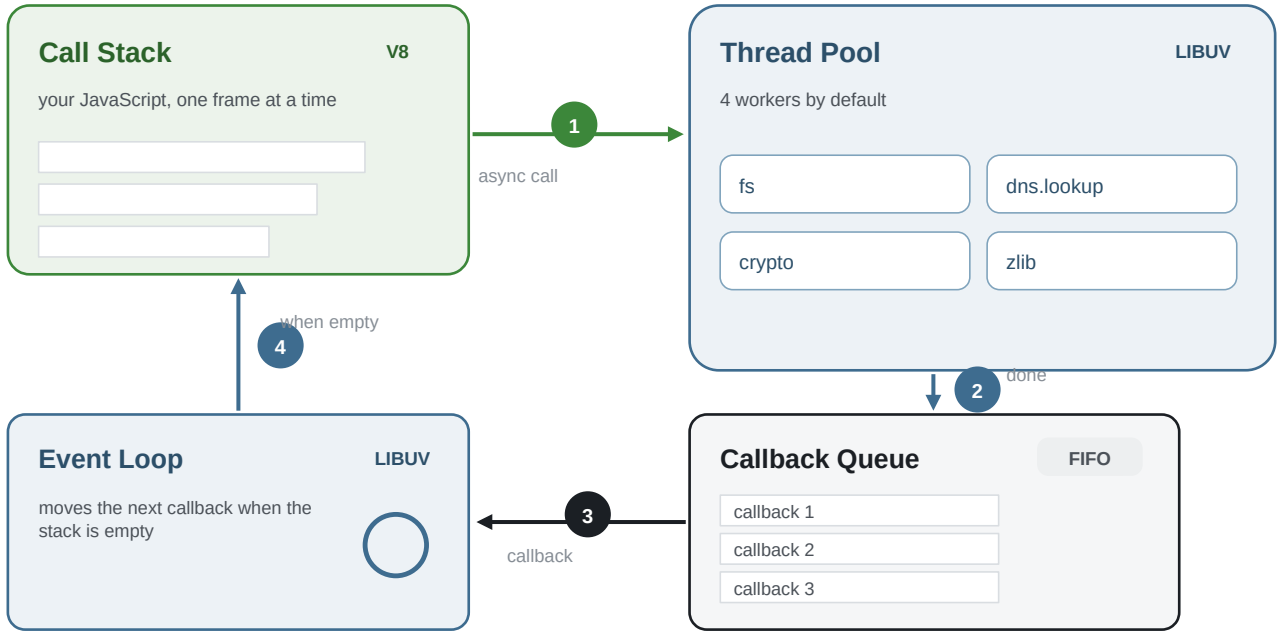
Tier up later

If add becomes hot, Sparkplug/TurboFan replace bytecode with native code.

This is the layer the JavaScript engine executes — pure V8, no I/O involved.

The event loop in one picture

ONE PROCESS · ONE JS THREAD



BROWSER WEB APIs → NODE THREAD POOL

1 V8

Call stack

Your JavaScript runs here, one frame at a time.

2 LIBUV

Thread pool

Blocking work waits here: fs, dns, crypto, zlib.

3 QUEUE

Callback queue

Finished callbacks wait until the stack is empty.

4 LOOP

Event loop

Moves the next callback back onto the stack.

V8 runs JavaScript. The pool does the blocking work. The loop moves the callback back.

Timers and sockets stay on the loop; the pool is only for blocking work.

Diagram concept: GeeksforGeeks — Event Loop in JavaScript (2026). Redrawn for Node.js.

DEMO · ASYNC FILE READ

One read, through the loop

read.mjs

```mjs
import { readFile } from 'node:fs';

console.log('1 · start');

readFile('./data.json', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('3 · loaded', data.length);
});

console.log('2 · end');
```

CONSOLE

1 · start → 2 · end → 3 · loaded

The file read happens between 2 and 3 — without blocking either.

1 V8

readFile(path, cb)

V8 runs the call on the stack. No I/O happens here.

2 NODE

binding → libuv request

Node hands the read to libuv.

3 LIBUV

thread pool reads the file

A worker performs the blocking syscall, then queues the callback.

4 V8

callback(err, data)

The loop moves it to the stack; V8 executes it.

The callback is not a new thread. Only the blocking read moved off the stack.

THREAD POOL · 01

What goes to the thread pool?

YES — BLOCKING WORK

fs	read, write, stat, open
dns.lookup	getaddrinfo for hostnames
crypto	pbkdf2, scrypt, randomBytes
zlib	compression and decompression

NO — THE LOOP HANDLES IT

sockets / HTTP	epoll, kqueue or IOCP — no worker parked
timers	setTimeout and setInterval
promises	microtasks inside V8



4 threads by default

UV_THREADPOOL_SIZE=8 node app.mjs

Four slow pool tasks fill every slot. A fifth fs call waits — even though the loop is idle.

Sockets use the OS readiness API; timers use the loop; promises use V8's microtask queue.

WORKLOAD · 01

CPU-bound vs I/O-bound

I/O-bound

Waiting dominates. The CPU is mostly idle.



green = V8 running · blue = libuv / kernel waiting

- HTTP calls, database queries, disk reads, DNS.
- Async APIs keep the loop free; the wait happens elsewhere.

HTTP · DB · disk · DNS

CPU-bound

Computing dominates. The loop cannot move.



green = V8 running; no idle gap, no I/O dispatched

- JSON.parse of a large payload, regex, image math, loops.
- Node cannot help by waiting: the work itself takes time.

JSON · regex · image · loops

I/O-bound: hide the wait.
Use async APIs and let libuv wait off the JS thread.

CPU-bound: move the work.
Worker threads for JS. The pool only helps for native work.

Native CPU work — crypto, zlib — can use the libuv pool. Pure JavaScript cannot.

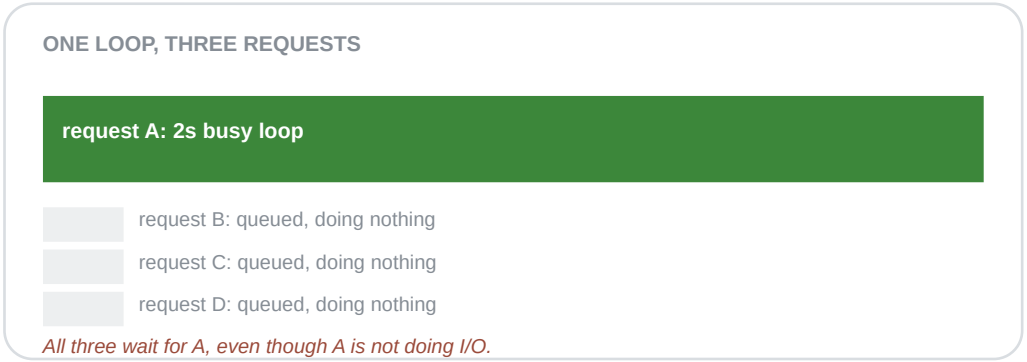
WORKLOAD · 02

Blocking the event loop

blocked.mjs

```mjs
import { readFileSync } from 'node:fs';
import { createServer } from 'node:http';

createServer((req, res) => {
  const data = readFileSync('./report.csv');
  const end = Date.now() + 2000;
  while (Date.now() < end) {}   // V8 never yields
  res.end(String(data.length));
}).listen(3000);
```



SYMPTOMS

- p99 latency spikes; health checks time out.
- Timers fire late; promises and callbacks queue up.
- CPU looks busy while throughput collapses.

DETECT IT

monitorEventLoopDelay()

node:perf_hooks — alert on max delay

node --prof app.mjs

profile, then --prof-process

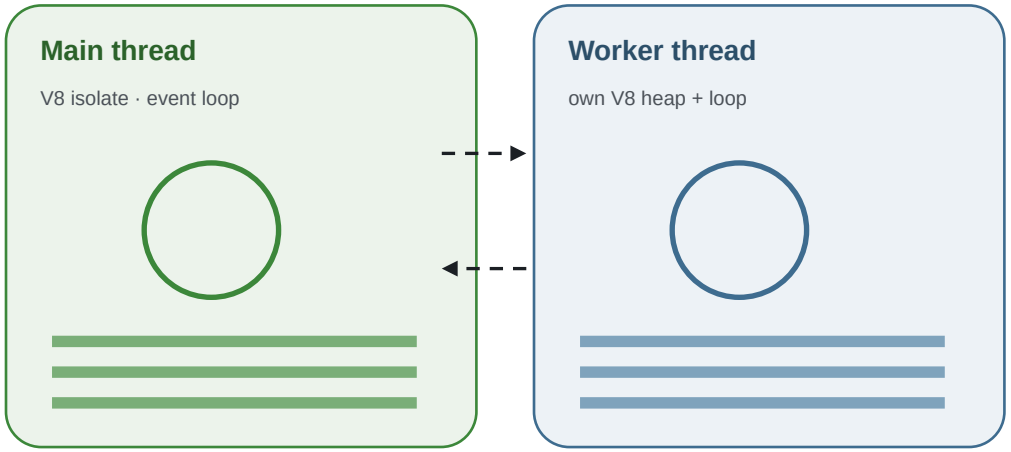
blocked-at / clinic

find the exact blocking stack

Never use a synchronous filesystem call or a busy loop in a request handler.

Worker threads: parallel JavaScript

SAME PROCESS · SEPARATE V8 ISOLATES



No shared JavaScript objects by default. Transfer ArrayBuffers or use SharedArrayBuffer + Atomics.

main.mjs

```mjs
import { Worker } from 'node:worker_threads';

const worker = new Worker(
  new URL('./fib-worker.mjs', import.meta.url),
  { workerData: 42 }
);

worker.on('message', console.log);
```

fib-worker.mjs

```mjs
import { parentPort, workerData } from 'node:worker_threads';

const fib = (n) =>
  n < 2 ? n : fib(n - 1) + fib(n - 2);

parentPort.postMessage(fib(workerData));
```

Use workers for CPU-bound JavaScript. Do not spawn one per request — pool them. I/O already has libuv; multi-core servers have cluster.

Each worker pays for its own isolate and heap. Spawn cost is real.

CHOOSING

Which tool for the job?

What kind of work is it?

I/O-BOUND

HTTP, DB, disk, DNS

Use the async API

Let the event loop and libuv wait.

BLOCKING NATIVE CALL

fs, getaddrinfo, pbkdf2

Use the thread pool

Already automatic; watch the 4 slots.

CPU-HEAVY JAVASCRIPT

parsing, hashing, image math

Use a worker thread

Keep the main loop free; pool workers.

MANY CORES / CONNECTIONS

API servers, WebSockets

Use cluster or processes

One event loop per core, shared port.

Start with async. Add a pool knob only when the pool is the bottleneck. Add workers only when the loop is. Add processes when you need cores.

WRAP-UP

What to remember from Part 2

01 **V8 executes. libuv waits.**

Your JavaScript always runs in V8, on the main thread. Async I/O is libuv.

02 **The loop is sacred.**

One slow handler stalls every connection. Measure event-loop delay.

03 **Match the tool to the work.**

Async for I/O, the pool for native blocking, workers for JS CPU, cluster for cores.

04 **Make it visible.**

Bytecode, profiles and loop-delay histograms beat intuition.

LAB · REPRODUCE IT

- node --print-bytecode -e "const add=(a,b)=>a+b; add(2,3)"
- UV_THREADPOOL_SIZE=1 node pool-demo.mjs
- node --prof app.mjs && node --prof-process isolate-*.log

Questions — and then we can reproduce the blocking demo live.